

PLSR项目开发学习笔记

目录

PLSR项目开发学习笔记

目录

- 一、项目目标和硬件
 - 二、工程模块划分
 - 三、board_profile板卡映射
 - 1. 为什么需要映射表
 - 2. 当前高速输出映射
 - 3. 查询接口
 - 4. X4和X5输入
 - 四、pulse_engine脉冲执行层
 - 1. 轴运行记录
 - 2. 启动接口
 - 3. 中断计数和停止
 - 4. 当前限制
 - 五、motion_planner运动规划层
 - 1. 参数结构
 - 2. 两层状态
 - 3. 初始化
 - 4. Start和Step分工
 - 5. 五种等待条件
 - 6. 段切换
 - 7. 当前限制
 - 六、PLSR寄存器映射
 - 1. 参数区
 - 2. 段参数区
 - 3. 状态区和控制区
 - 七、modbus_service通信服务
 - 1. 模块边界
 - 2. 文件阅读顺序
 - 3. 公共入口
 - 4. 寄存器区域分派
 - 5. 32位参数拆装
 - 6. 连续地址检查
 - 7. 三个功能码
 - 8. 异常和静默
 - 八、UART接收、T3.5和Modbus任务
 - 1. UART逐字节接收
 - 2. T3.5判断帧结束
 - 3. Modbus任务
 - 九、程序启动流程
 - 十、一条命令的完整数据流
 - 十一、调试顺序
 - 1. 脉冲输出调试
 - 2. 多段状态调试
 - 3. Modbus调试
 - 十二、常见问题
-

一、项目目标和硬件

本项目在XDM-60T4-E控制器内部的STM32F407IG6上实现一路PLSR功能。设备使用定时器PWM输出高速脉冲，使用普通输出点控制方向，使用X4和X5作为等待或外部触发信号，并通过USART1提供Modbus RTU从站通信。

硬件或软件	作用
XDM-60T4-E	运行STM32程序、uC/OS-II、PLSR和Modbus从站
STM32F407IG6	实际执行代码的MCU，系统时钟168 MHz
ST-Link	下载程序和在线调试
TG系列触摸屏或上位机	作为Modbus主站，配置参数和读取状态
IAR EWARM 8.40	编译和调试工程
STM32CubeMX	生成时钟、GPIO、UART和定时器初始化代码

PLSR输出和Modbus通信是两条不同的数据链路。

控制链路：触摸屏 -> RS-485/USART1 -> Modbus -> PLSR参数和命令
执行链路：PLSR参数 -> 运动规划 -> 定时器PWM -> 脉冲输出端子
反馈链路：定时器中断 -> 脉冲计数和状态 -> Modbus -> 触摸屏

ST-Link只负责下载和调试，不参与运行时Modbus通信。IAR关闭后，只要程序已经下载到Flash并且板卡供电正常，STM32仍然可以独立运行。

二、工程模块划分

应用代码位于 App 目录，硬件初始化和中断入口主要位于 Core 目录。

模块	主要职责	不负责的内容
board_profile.c/.h	Q点、定时器、GPIO和复用功能映射；X4/X5输入	不产生脉冲，不执行段表
pulse_engine.c/.h	配置PWM、输出固定频率和固定数量脉冲、累计计数	不决定下一段，不解析Modbus
motion_planner.c/.h	保存轴参数和段表，推进等待、跳转和多段状态	不直接解析RTU报文
modbus_service.c/.h	校验RTU帧、处理功能码、映射寄存器	不接收串口字节，不负责T3.5计时
app_tasks.c/.h	创建任务和信号量，连接Modbus任务与运动任务	不保存具体寄存器映射
main.c	初始化硬件，处理UART和定时器回调	不应堆放业务状态机

模块依赖关系如下。

```
main.c
-> app_tasks
    -> modbus_service
        -> motion_planner
            -> pulse_engine
                -> board_profile
```

依赖方向表示上层可以调用下层。pulse_engine 不应反过来调用 motion_planner，否则底层脉冲输出会依赖具体业务流程，后续复用和测试都会变困难。

三、board_profile板卡映射

1. 为什么需要映射表

Q0、Q1等是板卡端子名称，STM32真正使用的是GPIO端口、引脚、定时器实例、定时器通道和复用功能。若这些信息散落在脉冲代码中，切换端子时容易漏改。

BOARD_AXIS_CFG 把一个Q点需要的硬件信息放在一起。

```
typedef struct
{
    uint16_t      qPoint;
    GPIO_TypeDef  *port;
    uint16_t      pin;
    TIM_HandleTypeDef *htim;
    uint32_t      channel;
    uint32_t      af;
} BOARD_AXIS_CFG;
```

成员	作用
qPoint	板卡Q点编号，例如0表示Q0
port、pin	对应的GPIO端口和引脚
htim	CubeMX生成的定时器句柄
channel	PWM输出通道
af	GPIO复用功能编号

2. 当前高速输出映射

Q点	GPIO	定时器	通道
Q0	PF6	TIM10	CH1
Q1	PF8	TIM13	CH1
Q2	PF7	TIM11	CH1
Q3	PF9	TIM14	CH1

Q点	GPIO	定时器	通道
Q5	PE6	TIM9	CH2
Q6	PE5	TIM9	CH1
Q12	PH9	TIM12	CH2
Q15	PH6	TIM12	CH1
Q17	PB0	TIM3	CH3

需求规定PLSR脉冲端子只能在Y0至Y3中选择，因此业务层最终只应接受Q0至Q3。映射表可以保留其他高速点供底层测试或后续功能使用，但不能用“映射表中存在”代替业务参数范围检查。

3. 查询接口

```
const BOARD_AXIS_CFG *BoardProfileGetAxisByQ(uint16_t qPoint);
uint8_t BoardProfileIsHighSpeedQ(uint16_t qPoint);
```

BoardProfileGetAxisByQ() 遍历映射表，找到后返回配置地址，找不到返回空指针。返回的是 const 指针，调用者可以读取配置，但不应修改静态硬件映射。

4. X4和X5输入

输入点	STM32引脚	参数编码
X4	PB5	0
X5	PG12	1

XInputInit() 把两个引脚配置为输入，XInputIsHigh() 按参数编码读取电平。运动规划器不直接写GPIO寄存器，只通过该接口取得输入状态。

四、pulse_engine脉冲执行层

1. 轴运行记录

脉冲执行层为每个已登记Q点保存运行状态。

```
typedef struct
{
    const BOARD_AXIS_CFG *cfg;
    uint32_t target;
    volatile uint32_t sent;
    volatile uint8_t busy;
} PULSE_AXIS;
```

成员	作用
cfg	指向板卡映射信息

成员	作用
target	本次需要发送的脉冲周期数
sent	本次已经发送的脉冲周期数
busy	1表示该轴正在输出，0表示空闲

sent 和 busy 会在中断中修改，在任务中读取，因此使用 volatile 提醒编译器每次从内存读取。但 volatile 不等于线程同步，涉及多字节一致性时仍要考虑临界区。

2. 启动接口

```
uint8_t PulseEngineStartQ(uint16_t qPoint,  
                           uint32_t freqHz,  
                           int32_t pulseCount);
```

启动流程如下。

按Q点查找轴记录

- > 检查轴是否忙
- > 检查频率范围和脉冲个数
- > 判断定时器属于APB1还是APB2
- > 计算PSC和ARR
- > 设置CCR为半周期
- > 保存目标数量并清零本次计数
- > 清中断标志并允许更新中断
- > 启动PWM

频率计算代码先选择分频值，保证ARR不超过16位定时器上限。

```
psc = (timClk / freqHz + ARR_MAX) / (ARR_MAX + 1u);  
if (psc == 0u)  
{  
    psc = 1u;  
}  
arr = (timClk / (psc * freqHz)) - 1u;  
ccValue = (arr + 1u) / 2u;
```

传给HAL寄存器宏的PSC实际值为 psc - 1，因为硬件按寄存器值加1后分频。ARR同样按寄存器值加1形成一个完整周期。

3. 中断计数和停止

PulseEngineOnUpdate() 由定时器周期回调调用。每次更新把 sent 和累计值加1，到达目标后关闭更新中断并停止PWM。

```
HAL_TIM_PeriodElapsedCallback
-> 根据定时器实例确定Q点
-> PulseEngineOnUpdate(qPoint)
-> sent加1
-> totalSentCount加1
-> sent达到target时停止PWM并清busy
```

`PulseEngineStop()` 用于外部停止命令，不论目标是否到达都立即关闭输出。`PulseEngineIsBusy()` 供运动规划器判断当前段是否结束。

4. 当前限制

当前代码把负脉冲取绝对值后用于计数，但方向GPIO尚未接入脉冲启动流程。累计值也按无符号绝对数量增加，还不是可正可负的绝对位置。这两点会影响方向控制和绝对模式，不能只在注释中说明后就认为功能完成。

五、motion_planner运动规划层

1. 参数结构

`AXIS_MOTION_PARAM` 保存公共参数和10段段表，`SEGMENT_PARAM` 保存单段数据。

```
typedef struct
{
    uint32_t    freqHz;
    int32_t     pulseCount;
    WAIT_TYPE   waitType;
    uint16_t    waitTimeMs;
    uint16_t    actTimeMs;
    uint8_t     jumpSegment;
} SEGMENT_PARAM;
```

对外段号使用 1~10，数组下标使用 0~9，访问时需要减1。

```
segment = &motionParams.segment[curSegment - 1u];
```

2. 两层状态

运动规划器有轴运行状态和段内部状态两层。

状态类型	状态	含义
<code>RUN_STATE</code>	<code>IDLE</code>	空闲，等待启动
<code>RUN_STATE</code>	<code>RUNNING</code>	正在执行段表
<code>RUN_STATE</code>	<code>DONE</code>	本轮完成，可以再次启动
<code>SECTION_STATE</code>	<code>SENDING</code>	当前段正在发送脉冲
<code>SECTION_STATE</code>	<code>WAITING</code>	当前段脉冲已完成，等待条件满足

运行状态回答“整条指令处于什么阶段”，段内部状态回答“当前这一段是在发脉冲还是等待”。将两者混在一个枚举中，会导致状态数量快速膨胀。

3. 初始化

`MotionPlannerInit()` 设置默认参数、清空段表状态，并记录EXT输入初始电平。

公共参数设默认值

-> 10段频率设默认值，脉冲个数清零

-> runState设为IDLE

-> curSegment设为0

-> sectionState设为SENDING

-> 读取EXT输入初始电平

初始化默认值只用于让结构体处于确定状态。真正启动前仍要检查总段数、起始段、频率和脉冲个数是否合法。

4. Start和Step分工

`MotionPlannerStart()` 是事件触发入口，只调用一次。`MotionPlannerStep()` 是周期推进入口，需要由任务反复调用。

函数	调用方式	主要工作
<code>MotionPlannerStart()</code>	收到启动命令时调用一次	检查状态和参数，设置起始段，启动第一段
<code>MotionPlannerStep()</code>	任务中周期调用	检查脉冲完成、输入信号、等待时间和段切换
<code>MotionPlannerStop()</code>	收到停止命令时调用	停止PWM，清当前段，回到空闲态

5. 五种等待条件

发送态先检查可提前切换的条件，再检查脉冲是否完成。

SENDING

-> ACT时间到：切换下一段

-> EXT出现边沿：切换下一段

-> EXT_OR_DONE出现边沿：切换下一段

-> 脉冲仍忙：保持发送态

-> 脉冲完成：进入WAITING或直接切换

等待态处理WAIT时间和WAIT信号。

WAITING

-> WAIT_TIME：等待时间到后切换

-> WAIT_SIGNAL：输入电平有效后切换

使用 `nowTick - startTick >= delay` 判断超时，可以正确处理32位毫秒计数自然回绕，不能改成 `nowTick >= startTick + delay` 后忽略溢出。

6. 段切换

SectionAdvance() 先保存完成段号，再读取完成段的跳转编号。

```
doneSeg = curSegment;
jumpSeg = motionParams.segment[doneSeg - 1u].jumpSegment;
```

跳转编号为0时顺序加1，非0时进入指定段。目标超过总段数时结束整轮执行。段切换后重新记录时间和EXT输入电平，避免把上一段的边沿带入下一段。

7. 当前限制

当前运动规划器使用固定的PULSE_QPOINT启动Q0，尚未把pulseTerminal和方向端子完整接入。加减速模式、相对绝对模式和发送模式已经有参数字段，但主执行流程尚未实现对应语义。

六、PLSR寄存器映射

1. 参数区

地址	参数	数据类型
0x1000	脉冲端子，0至3对应Y0至Y3	16位
0x1001	方向端子，0至3对应Y12至Y15	16位
0x1002	WAIT信号，0对应X4，1对应X5	16位
0x1003	EXT信号，0对应X4，1对应X5	16位
0x1004	发送模式，0完成，1后续	16位
0x1005	方向延时，单位毫秒	16位
0x1006	方向逻辑，0正逻辑，1负逻辑	16位
0x1007	加减速模式，0直线，1 S曲线，2正弦	16位
0x1008	运行模式，0相对，1绝对	16位
0x1009	总段数，1至10	16位
0x100A	起始段	16位
0x100B~0x100C	默认速度	32位，低字在前
0x100D~0x100E	起始速度	32位，低字在前
0x100F	空档，不使用	读取为0，禁止写入
0x1010~0x1011	终止速度	32位，低字在前
0x1012	加速时间	16位
0x1013	减速时间	16位

2. 段参数区

第N段基址为 $0x1100 + (N - 1) * 0x10$ 。

偏移	参数
0~1	频率，32位
2~3	脉冲个数，32位有符号数
4	等待条件
5	WAIT时间
6	ACT时间
7	跳转编号
8~15	空档

例如第3段基址为 `0x1120`，频率位于 `0x1120~0x1121`，脉冲个数位于 `0x1122~0x1123`，跳转编号位于 `0x1127`。

3. 状态区和控制区

地址	权限	内容
<code>0x2000~0x2001</code>	只读	累计脉冲数
<code>0x2002~0x2003</code>	只读	当前段频率
<code>0x2004</code>	只读	运行状态
<code>0x2005</code>	只读	当前段
<code>0x2006</code>	只读	错误码，当前实现返回0
<code>0x3000</code>	只写	bit0启动、bit1停止、bit2清累计

控制寄存器是命令入口，不保存成普通变量。主站写入后立即执行对应动作，后续读取该地址应返回非法地址。

七、modbus_service通信服务

1. 模块边界

`modbus_service.c` 输入一帧已经由T3.5确认完成的RTU请求，输出正常应答、异常应答或0长度。

它负责：

- CRC和站号检查
- 功能码分发
- 寄存器地址映射

- 参数、状态和控制访问
- 构造正常应答和异常应答

它不负责：

- UART逐字节接收
- T3.5帧结束定时
- 任务创建和信号量等待
- 实际产生PWM波形

2. 文件阅读顺序

当前文件通过调整函数定义顺序取消了内部前向声明，按依赖从基础到入口排列。

```
ModbusServiceCalcCrc
-> ModbusServiceCopyFrame
-> GetRegisterArea
-> 各区域读写函数
-> ReadRegister和WriteRegister
-> ValidateRegisterRange
-> BuildExceptionResponse
-> 三个功能码处理函数
-> ModbusServiceProcessFrame
```

这样阅读到一个函数时，它调用的内部函数通常已经在上方出现。头文件只保留真正给其他模块调用的公共接口。

3. 公共入口

```
uint16_t ModbusServiceCopyFrame(uint8_t *dstBuf,
                                const uint8_t *srcBuf,
                                uint16_t len);

uint16_t ModbusServiceProcessFrame(uint8_t *rxBuf,
                                    uint16_t rxLength,
                                    uint8_t *txBuf);

uint16_t ModbusServiceCalcCrc(const uint8_t *data, uint16_t len);
```

`CopyFrame()` 检查长度后复制完整请求。`ProcessFrame()` 完成公共校验和功能码分发。`CalcCrc()` 既供协议入口校验请求，也供各功能码构造应答。

4. 寄存器区域分派

`GetRegisterArea()` 只判断地址属于参数区、段参数区、状态区、控制区还是非法区域。

```
地址
-> 参数区: ReadParamRegister或WriteParamRegister
-> 段参数区: ReadSegmentRegister或WriteSegmentRegister
-> 状态区: 只允许ReadStatusRegister
-> 控制区: 只允许WriteControlRegister
-> 其他: 非法地址
```

区域判断和具体参数读写分开后，顶层流程不需要知道每个地址对应哪个结构体成员。

5. 32位参数拆装

参数表规定低地址保存低16位，高地址保存高16位。写低字时保留原高字，写高字时保留原低字。

```
param->defaultFreqHz = (param->defaultFreqHz & 0xFFFF0000u) | value;

param->defaultFreqHz = (param->defaultFreqHz & 0x0000FFFFu) |
    ((uint32_t)value << 16u);
```

有符号脉冲数先按 `uint32_t` 保存原始位模式，分别替换高低16位，最后再转换回 `int32_t`。这样负数的补码不会因移位符号扩展而被破坏。

6. 连续地址检查

`ValidateRegisterRange()` 在真正执行读写前检查整个地址区间。

写请求需要拒绝：

- 状态区地址
- 不存在的地址
- 跨多个寄存器写控制区
- 参数区空档 `0x100F`
- 每段偏移 `8~15` 的空档

读参数区的 `0x100F` 返回0，读段内空档也返回0，便于主站进行整块读取。写空档会返回非法地址，避免主站误以为配置已经生效。

7. 三个功能码

功能码	处理函数	作用
0x03	<code>HandleReadHoldingRegisters()</code>	读取连续保持寄存器
0x06	<code>HandlewriteSingleRegister()</code>	写一个保持寄存器，成功时回显请求
0x10	<code>HandlewriteMultipleRegisters()</code>	写多个保持寄存器，成功时回显起始地址和数量

Modbus报文中的16位地址、数量和值按高字节在前组合。

```
startAddress = ((uint16_t)request[2] << 8u) | request[3];
registerCount = ((uint16_t)request[4] << 8u) | request[5];
```

读取应答同样按高字节、低字节写入。

```
response[3u + index * 2u] = (uint8_t)(value >> 8u);
response[4u + index * 2u] = (uint8_t)(value & 0xFFu);
```

8. 异常和静默

情况	处理
请求长度小于4字节	静默丢弃
CRC错误	静默丢弃
站号不是本机地址	静默丢弃
不支持的功能码	异常 01
地址或权限非法	异常 02
数量、长度或Byte Count错误	异常 03

异常应答格式为：

```
从站地址 + 原功能码 | 0x80 + 异常码 + CRC低字节 + CRC高字节
```

CRC在RTU帧尾低字节先发送，但寄存器数据本身仍然高字节先发送。

八、UART接收、T3.5和Modbus任务

1. UART逐字节接收

USART1使用中断方式一次接收一个字节。每次进入 `HAL_UART_RxCpltCallback()` 时，把字节放入全局接收缓冲区，然后重启TIM2帧间隔定时器。

```
USART1收到1字节
-> 保存到rxBuf[rxCount]
-> rxCount加1
-> 停止TIM2并清零计数器
-> 清TIM2更新标志
-> 再次启动TIM2
-> 重新启动下一字节接收
```

缓冲区满时把 `rxCount` 清零，表示丢弃当前异常帧并重新开始。正式诊断还应增加溢出计数，便于区分主站超时和从站接收溢出。

2. T3.5判断帧结束

最后一个字节后不再有新字节重启TIM2，定时器达到设定的静默时间后进入更新中断。

```
if (htim->Instance == TIM2)
{
    __HAL_TIM_DISABLE(&htim2);
    OSemPost(ModbusSemGet());
}
```

中断只停止定时器并发布信号量，不执行CRC和寄存器访问。这样可以缩短中断占用时间。

3. Modbus任务

ModbusTask() 永久等待帧就绪信号量。

OSSemPend等待完整帧

-> ModbusServiceCopyFrame复制请求

-> ModbusServiceProcessFrame解析并执行

-> 应答长度大于0时调用HAL_UART_Transmit

-> rxCount清零

-> 再次等待

任务收到信号量后才运行，没有完整帧时处于阻塞状态，不需要固定周期轮询。

当前接收缓冲区由中断写入、任务复制和清零。后续完善时应加入短临界区或双缓冲，避免任务复制期间新字节进入造成竞态。

九、程序启动流程

程序启动时应先完成硬件初始化，再初始化应用模块和操作系统任务。

顺序	主要动作	作用
1	HAL_Init()	初始化HAL和基础时基
2	SystemClock_Config()	配置系统时钟
3	GPIO、TIM、USART初始化	准备输入、PWM、帧定时和通信硬件
4	XInputInit()	初始化X4和X5输入
5	MotionPlannerInit()	初始化公共参数、段表和状态机
6	启动USART1单字节接收	准备接收Modbus请求
7	OSInit()	初始化uC/OS-II
8	AppTasksCreate()	创建任务和Modbus帧信号量
9	OSStart()	启动任务调度

OSStart() 之后，业务运行在任务和中断中。主函数不再承担周期业务循环。

十、一条命令的完整数据流

以主站写 0x3000=0x0001 启动为例。

触摸屏发送0x06写单寄存器请求

-> USART1逐字节接收

-> TIM2静默超时发布信号量

-> ModbusTask复制完整请求

-> ModbusServiceProcessFrame检查CRC和站号

-> HandlewriteSingleRegister解析地址和值

```
-> ValidateRegisterRange确认0x3000允许单写
-> WriteRegister识别控制区
-> WriteControlRegister检查bit0
-> MotionPlannerStart检查段表并启动起始段
-> PulseEngineStartQ配置定时器PWM
-> 定时器更新中断逐个统计脉冲
-> MotionPlannerStep检查当前段完成和等待条件
-> 段表全部完成后runState变为DONE
-> 主站通过0x03读取0x2004和0x2005观察状态
```

这条链路跨越通信、状态机和硬件三层。调试时应逐层确认，不应看到“主站超时”就直接修改PWM代码。

十一、调试顺序

1. 脉冲输出调试

检查项	正常现象
Q点映射	查询Q0能得到TIM10 CH1配置
启动参数	频率和脉冲数合法时返回1
示波器频率	与设定频率接近
PWM数量	到达目标数量后停止
busy状态	启动后为1，停止后为0
累计计数	每个更新周期增加1

2. 多段状态调试

先使用全部WAIT时间条件，排除外部输入干扰。确认顺序执行后，再逐项加入跳转、WAIT信号、ACT时间、EXT信号和EXT_OR_DONE。

```
单段
-> 两段顺序
-> 五段顺序
-> 指定段跳转
-> WAIT时间
-> WAIT信号
-> ACT时间
-> EXT边沿
-> EXT或完成
```

3. Modbus调试

先用固定请求验证CRC和站号，再验证寄存器映射。

测试	预期结果
正确CRC读取 0x1000	返回正常 0x03 应答
错误CRC	从站静默，无应答
错误站号	从站静默，无应答
不支持功能码	返回异常 01
读取不存在地址	返回异常 02
写 0x100F	返回异常 02
0x10 的Byte Count错误	返回异常 03
写入32位频率后回读	高低字组合与原值一致
写 0x3000 bit2	累计脉冲清零

十二、常见问题

现象	优先检查
没有脉冲输出	Q点映射、定时器PWM通道、频率和脉冲数、轴busy状态
频率不正确	APB定时器时钟、PSC、ARR和示波器测量方式
数量多一个或少一个	更新中断计数起点、标志清除和停止时机
多段跳过某一段	段号与数组下标、是否先加段号再读跳转参数
EXT进入新段后立即触发	是否在段开始记录当前输入电平，是否使用边沿检测
Modbus完全无应答	串口参数、站号、CRC、T3.5定时器和信号量
返回异常 02	地址、权限、段内空档和 0x100F 空档
返回异常 03	数量、帧长度、Byte Count和协议上限
32位参数回读错误	寄存器高低字顺序和单寄存器字节顺序
启动命令有应答但不动作	段表参数、起始段、脉冲数是否为0、运动状态是否正在运行
清累计后又出现旧值	是否仍有脉冲中断继续累加，清零和停止命令顺序是否合理
修改参数但执行没有变化	参数是否只写入结构体，实际执行流程是否读取并使用该字段